

Machine Learning Foundations (MLF)

Week 1 • Session 1 — Foundations of Machine Learning, Data Representations & Paradigms

Lead Instructor: Karthik Thiagarajan (IIT Madras Alumnus & Senior Course Instructor)
Co-Instructor: Prashant | **Teaching Assistants:** Srinivasan, Dhruvi

Session Format: Live Interactive Lecture (~2h 55m)
YouTube URL: youtube.com/live/B28PSkbZzZM
Subject Scope: Core Mathematical Foundations for MLT & MLP

[INSTRUCTOR INSIGHT] Course Positioning & Syllabus Trajectory

MLF (Machine Learning Foundations) serves as the mathematical and theoretical backbone for the entire Data Science curriculum at IIT Madras. It precedes and directly enables **MLT (Machine Learning Techniques)** and **MLP (Machine Learning Practice)**.

- Week 1:** Introduction to ML, Data Representations, Supervised vs. Unsupervised Paradigms, Loss Formulation.
- Weeks 2–5:** Rigorous Linear Algebra (Vectors, Matrices, Vector Spaces, Projections, Eigenvalues, SVD, PCA).
- Week 3:** Unsupervised Clustering (K-Means algorithm and distance metrics).
- Weeks 6–8:** Multivariate Calculus & Gradient Formulations.
- Weeks 9–10:** Convex Optimization, Duality, and Gradient Descent.

Core Teaching Philosophy: Rather than treating ML models as empirical black boxes, MLF teaches you to see models as rigorous optimization problems defined over linear algebraic vector spaces.

1. THE MACHINE LEARNING PARADIGM SHIFT

1.1 What is Machine Learning?

Machine Learning is fundamentally defined as **learning from data and past experience** to solve tasks without being explicitly programmed with deterministic rules. The discipline is deeply rooted in **Pattern Recognition** — extracting statistical regularities and latent structures from observational data. A foundational reference recommended by the instructor is *Pattern Recognition and Machine Learning (PRML)* by Christopher M. Bishop (Microsoft Research).

1.2 Procedural (Rule-Based) Programming vs. Machine Learning

To appreciate why Machine Learning exists, we must examine the limitations of classical software engineering:

Dimension	Traditional / Procedural Programming	Machine Learning Paradigm
Core Logic	Human programmer writes explicit, hand-crafted conditional rules (if-else, loops, mathematical formulas).	Learning algorithm searches for and learns the mapping function $f : X \rightarrow Y$ directly from data.
Inputs Provided	Data + Explicit Rules	Data + Observed Outputs (Labels)
Produced Result	Answers / Output Actions	Learned Model / Rules / Mapping Function f
When to Use?	When problem logic is well-understood (e.g., payroll calculation, sorting algorithms, accounting).	When exact analytical rules are unknown, intractable, or too complex to hand-code (e.g., computer vision, speech, fraud detection).

[KEY CONCEPT] Why Can't We Hand-Code Computer Vision or Speech?

Consider identifying handwritten digits (MNIST). A human can write a digit '4' in thousands of ways: open top, closed top, tilted, thick stroke, thin stroke, varying aspect ratios. Writing deterministic if-else rules on pixel coordinates is practically impossible because slight translations or lighting changes break hand-crafted heuristics. Machine learning replaces fragile heuristics with a continuous parameterized function optimized over thousands of sample images.

2. DATA REPRESENTATIONS & THE UNIVERSAL MATRIX FRAMEWORK

2.1 Diverse Real-World Data Modalities

Real-world data originates in heterogeneous formats. A vital preliminary step in ML is transforming raw sensory modalities into standardized mathematical structures:

- **Tabular / Structured Data:** Organized in relational tables with defined schemas. Rows correspond to individual records/observations (n samples); columns represent measured properties, attributes, or features (d dimensions).
- **Image Data:** 2D or 3D numerical pixel arrays. Grayscale images are 2D matrices (height $\times$ width) where each entry is a pixel brightness value (0 to 255). In the **MNIST** dataset, each handwritten digit is a 28×28 grayscale image, which is flattened into a **784-dimensional vector** in $\mathbf{R}^{784}$. Color images include 3 channels (RGB), forming 3D tensors.
- **Audio / Speech Signals:** 1D continuous pressure waveforms sampled at discrete intervals (e.g., 16 kHz or 44.1 kHz), frequently transformed into 2D time-frequency spectrograms via Fourier transforms or discretized MFCC feature vectors.
- **Text / Natural Language:** Sequences of symbolic words or sub-word tokens mapped into dense continuous vector spaces (Word2Vec, GloVe, Transformer embeddings) where semantic similarity corresponds to geometric proximity.

2.2 The Linear Algebra Perspective: Data as Matrices

A central pillar emphasized throughout MLF is that **all data is ultimately represented as matrices and vectors over the real numbers $\mathbf{R}$** . A dataset consisting of n observations, each described by d features, forms a **Design Matrix** $X \in \mathbf{R}^{n \times d}$:

[MATHEMATICAL FORMULATION] The Core Mathematical Mapping

Design Matrix $X \in \mathbf{R}^{n \times d}$:

Row i represents the i -th observation vector: $\mathbf{x}_i^T = [x_{i,1}, x_{i,2}, \dots, x_{i,d}] \in \mathbf{R}^d$

Column j represents the j -th measured attribute across all n instances.

Target Label Vector $\mathbf{y} \in \mathbf{R}^n$ (in supervised learning): $\mathbf{y} = [y_1, y_2, \dots, y_n]^T$

[INSTRUCTOR INSIGHT] Karthik Sir's Golden Intuitive Anchor

"It is very easy to get lost in all the abstract mathematics during the linear algebra weeks. If you find yourself lost in the middle of the course, I want you to remember: At some level, a matrix is nothing but a dataset! Rows represent individual records; columns represent their properties or features. Matrix multiplication is transforming or projecting data; solving linear systems is finding optimal parameter combinations."

[KEY CONCEPT] Feature Representation & Vectorization Principles

Before feeding data into mathematical models, raw features must be properly vectorized:

- **Continuous Features:** Real numbers such as area, income, or temperature mapped directly onto $\mathbf{R}$.
- **Categorical Features:** Discrete categories (e.g., city, gender, color) must be encoded as one-hot binary vectors to prevent false ordinal rankings.
- **Feature Scaling:** Normalization (min-max scaling into $[0, 1]$) or standardization (zero mean, unit variance) ensures that features with large numerical magnitudes (like income in Lakhs) do not dominate optimization over smaller features (like age).

3. MACHINE LEARNING PARADIGMS: UNSUPERVISED VS. SUPERVISED

3.1 Unsupervised Learning: Discovering Latent Structure

In **Unsupervised Learning**, the learning algorithm is provided solely with input feature vectors $D = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$. There is no external teacher or supervisor providing target labels y . The primary objective is to uncover intrinsic patterns, manifolds, clusterings, or low-dimensional projections.

E-Commerce Case Study (Customer Segmentation): Platforms like Amazon describe millions of customers by attributes such as [Annual Income, Age, Browsing Frequency]. Clustering algorithms (introduced in Week 3) group shoppers into coherent behavioral cohorts (e.g., budget-conscious students, high-income tech professionals) to enable targeted product recommendations without requiring manual tags.

3.2 Supervised Learning: Learning with a Teacher

In **Supervised Learning**, supervision is available in the form of ground truth labels: $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$. The supervisor acts as an instructor evaluating the model's predictions. The goal is to estimate a parameterized mapping function $f : X \rightarrow Y$ that accurately predicts unseen labels $y_{\text{new}} \approx f(\mathbf{x}_{\text{new}})$.

Attribute	Unsupervised Learning	Supervised Learning
Training Data	Only features provided: $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \mathbf{x}_i \in \mathbb{R}^d$. No target labels y exist.	Feature-label pairs provided: $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$. Ground truth labels y are known.
Primary Goal	Discover underlying patterns, clusterings, manifold geometries, or low-dimensional representations.	Learn a generalizable predictive mapping function $f : X \rightarrow Y$ such that $f(\mathbf{x}_{\text{new}}) \approx y_{\text{new}}$.
Core Problem Types	• Clustering: Grouping similar data points. • Dimensionality Reduction: PCA, SVD compression. • Density Estimation: Modeling underlying distributions.	• Regression: Predicting continuous real values ($y \in \mathbb{R}$). • Classification: Predicting categorical labels ($y \in \{1, \dots, K\}$).
Real-World Example	Customer Segmentation on Amazon using features like [Income, Age] to find shopper cohorts.	Real Estate Valuation (predicting house price from area/rooms) or Spam Detection .
Data Acquisition Cost	Low to moderate: Raw data is abundant and does not require human manual annotation.	High: Labeling requires extensive human labor (crowdsourcing via Amazon Mechanical Turk, domain experts).

[KEY CONCEPT] The Data Labeling Bottleneck & Amazon Mechanical Turk

Supervised learning is powerful, but obtaining reliable ground truth labels is expensive. The lecture highlights platforms like **Amazon Mechanical Turk (MTurk)**, which deploy crowdsourced global workforces to label images, verify text, and transcribe speech. In specialized domains like medicine (radiology scans) or manufacturing (detecting micro-scratches on manufactured parts), labels cannot be crowdsourced — they require expensive domain experts.

[INSTRUCTOR INSIGHT] Ground Truth & Label Noise in Real-World Applications

Even when supervision is present, real-world labels are rarely pristine. Human annotators make errors, medical specialists disagree on diagnoses, and sensor calibrations drift. Understanding label uncertainty and building models that are robust to noisy labels is a fundamental theme explored throughout advanced machine learning.

4. SUPERVISED LEARNING DEEP-DIVE: REGRESSION

4.1 Problem Definition & Mathematical Setup

In **Regression**, the target label y is a **continuous real number** ($y \in \mathbf{R}$).

Running Example: Housing price prediction in Mumbai.

- **Input Features** $\mathbf{x} \in \mathbf{R}^d$: x_1 = Carpet Area (sq. ft.), x_2 = Number of Bedrooms, x_3 = Age of Construction (years), x_4 = Proximity to transit.
- **Target Label** $y \in \mathbf{R}$: Historical Selling Price (in Lakhs INR).
- **Model Hypothesis**: A function $f: \mathbf{R}^d \rightarrow \mathbf{R}$ that outputs predicted price $y_{\text{pred}} = f(\mathbf{x})$.

4.2 The Evolution of Regression Loss Functions

For a dataset of n houses, let $e_i = y_i - f(\mathbf{x}_i)$ denote the prediction error (residual) on the i -th house. How should we define total loss over all houses? The lecture provides a step-by-step mathematical progression:

1. Raw Error Sum: $\sum_{i=1}^n (y_i - f(\mathbf{x}_i))$ — **Fails completely!**

If house 1 has error +10 Lakhs (under-prediction) and house 2 has error -10 Lakhs (over-prediction), their sum is 0. The model made two huge errors, but the metric reports zero error because positive and negative residuals cancel each other out.

2. Mean Absolute Error (MAE): $(1/n) \sum_{i=1}^n |y_i - f(\mathbf{x}_i)|$ — **Intuitive, but non-smooth!**

Taking the absolute value avoids sign cancellation and is robust to outliers. *Major Mathematical Drawback:* The modulus function $|u|$ is **not differentiable at $u = 0$** . Its derivative jumps abruptly from -1 to +1. Because optimization algorithms like Gradient Descent depend on well-defined gradients, non-smooth loss functions introduce mathematical complications.

3. Sum of Squared Errors (SSE): $\sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2$ — **Smooth, but size-dependent!**

Squaring residuals eliminates signs, penalizes large mistakes severely, and is **everywhere smooth and continuously differentiable**. *Limitation:* SSE scales linearly with dataset size n . A model tested on 1,000 houses will have a vastly higher SSE than on 10 houses, even if its per-house accuracy is identical.

4. Mean Squared Error (MSE): $(1/n) \sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2$ — **The Standard Metric!**

Dividing by n yields the average expected squared error per sample, making performance comparable across datasets of varying sizes while preserving smoothness.

4.3 Hypothesis Space & Linear Models

We cannot search across the set of all arbitrary mathematical functions (which is infinite and unconstrained). Instead, we restrict our search to a parametrized **hypothesis class H** . The simplest and most fundamental class is the **linear family**:

[MATHEMATICAL FORMULATION] Linear Model Formulation

Linear Regression Hypothesis:

$$f(\mathbf{x}; \mathbf{w}, b) = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{w} = [w_1, \dots, w_d]^T \in \mathbf{R}^d$ are feature weights, and $b \in \mathbf{R}$ is the bias/intercept.

Optimization Objective (Empirical Risk Minimization):

$$\min_{\mathbf{w}, b} \text{MSE}(\mathbf{w}, b) = (1/n) \sum_{i=1}^n [y_i - (\mathbf{w}^T \mathbf{x}_i + b)]^2$$

The role of the **learning algorithm** is to search parameter space $(\mathbf{w}, b)$ to find optimal parameters $(\mathbf{w}^*, b^*)$ minimizing this loss.

[KEY CONCEPT] Why MSE Yields a Convex Quadratic Optimization Problem

When $f(\mathbf{x})$ is linear in parameters $(\mathbf{w}, b)$, the MSE loss function forms a convex multi-dimensional paraboloid (bowl shape). Because it is strictly convex, it has a **unique global minimum** with no local minima! In Week 4, we will derive the exact closed-form solution: $\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}$ (the famous Normal Equations of Linear Algebra).

5. SUPERVISED LEARNING DEEP-DIVE: CLASSIFICATION & CONVEX SURROGATES

5.1 Problem Definition & Decision Rule

In **Classification**, the target label y belongs to a **discrete set of categorical classes**.

- **Binary Classification:** Labels are two-valued, conventionally encoded as $y \in \{0, 1\}$ or $y \in \{-1, +1\}$ (e.g., Spam vs. Ham; Defective vs. Intact manufacturing parts).
- **Multi-Class Classification:** Labels take one of K discrete values, $y \in \{0, 1, \dots, K-1\}$ (e.g., MNIST digits 0 through 9).

For a binary linear classifier with $y \in \{-1, +1\}$, we compute a linear score $g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, and apply a threshold:

Decision Rule: $y_{\text{pred}} = \text{sign}(\mathbf{w}^T \mathbf{x} + b) = +1$ if $\mathbf{w}^T \mathbf{x} + b \geq 0$, else -1 .

5.2 The Zero-One Loss (0-1 Loss) & Its Fatal Flaw

The natural metric for classification performance is counting the number of mistakes made by the classifier:

[MATHEMATICAL FORMULATION] The Core Theoretical Bottleneck

Zero-One Loss Definition:

$$L_{0-1}(y, y_{\text{pred}}) = I(y \neq y_{\text{pred}}) = \begin{cases} 0 & \text{if } y = y_{\text{pred}} \text{ (correct)} \\ 1 & \text{if } y \neq y_{\text{pred}} \text{ (mistake)} \end{cases}$$

Empirical Classification Error: $R_{0-1}(f) = (1/n) \sum_{i=1}^n I(y_i \neq \text{sign}(\mathbf{w}^T \mathbf{x}_i + b))$

Why Gradient Descent CANNOT Optimize 0-1 Loss:

- The 0-1 loss function is a **piecewise constant step function**.
- Its derivative with respect to weights $\mathbf{w}$ is **ZERO almost everywhere**, and undefined/infinite at the boundary.
- Gradient descent relies on gradient slopes: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$. If $\nabla L = \mathbf{0}$, the weights never update!
- Directly minimizing 0-1 loss is known to be an **NP-hard combinatorial optimization problem**.

5.3 The Solution: Convex Surrogates

Because 0-1 loss is intractable for gradient optimization, machine learning introduces **Convex Surrogate Loss Functions** — continuous, convex functions that upper-bound the 0-1 loss and possess well-defined gradients everywhere:

Loss Function	Mathematical Formulation	Primary Algorithm / Context
Hinge Loss	$L(y, g(\mathbf{x})) = \max(0, 1 - y \cdot g(\mathbf{x}))$	Support Vector Machines (SVM) — enforces a margin of separation ≥ 1 .
Perceptron Loss	$L(y, g(\mathbf{x})) = \max(0, -y \cdot g(\mathbf{x}))$	Rosenblatt's Perceptron — penalizes only misclassified samples.
Logistic Loss (Binary Cross-Entropy)	$L(y, g(\mathbf{x})) = \log(1 + \exp(-y \cdot g(\mathbf{x})))$	Logistic Regression & Neural Networks — smooth, differentiable, outputs calibrated probabilities.
Categorical Cross-Entropy	$L = -\sum_{k=1}^K y_k \log(p_k)$	Multi-Class Classification ($K > 2$) using Softmax probabilities.

[KEY CONCEPT] Why Convexity is Paramount (Preview of Weeks 9 & 10)

A function is **convex** if any line segment connecting two points on its graph lies above or on the graph. The critical property for optimization is:
For a convex function, every local minimum is guaranteed to be a global minimum! There are no deceptive local valleys or saddle points to trap gradient descent. This is why replacing 0-1 loss with convex surrogates is the foundation of modern machine learning.

6. THE COMPLETE MACHINE LEARNING LIFECYCLE

Building a machine learning system is an iterative engineering lifecycle consisting of nine distinct phases discussed in detail during the session:

Phase	Stage Name	Key Activities & Concepts Discussed
1	Problem Formulation	Define business objectives. Categorize learning setup: Supervised (regression vs. classification) or Unsupervised.
2	Data Acquisition	Gather data from historical databases, web scraping, API feeds, surveys, sensors, or crowdsourced labeling (MTurk).
3	Data Preprocessing & Cleaning	Real-world data is dirty. Handle missing values (imputation), detect measurement errors, handle outliers, normalize/scale features.
4	Feature Engineering & Representation	Convert raw modalities (images, audio, text, categorical attributes) into numerical vectors in $\mathbf{R}^d$.
5	Model Selection	Choose hypothesis class: Linear models, Polynomial regression, Decision trees, Support Vector Machines, Neural Networks.
6	Loss Function Definition	Define a smooth/convex objective function (MSE for regression; Hinge/Logistic cross-entropy for classification).
7	Optimization & Training	Execute optimization algorithms (Gradient Descent, Convex Solvers) to learn optimal parameters $\mathbf{w}^*$ from training data.
8	Evaluation & Validation	Rigorous performance testing using clean Train / Validation / Test data splits (see Academic Analogy below).
9	Deployment & Monitoring	Deploy model to cloud serving infrastructure. Continually monitor for Data Drift and Concept Drift .

[INSTRUCTOR INSIGHT] The Academic Analogy for Train, Validation, and Test Sets

Karthik Sir shared an intuitive analogy that clarifies data splitting:

- **Training Data = Textbook & Homework Assignments:** You use these to learn concepts, make mistakes, and adjust your knowledge (model parameter updates).
- **Validation Data = Mock Exams / Pre-Boards:** You take mock tests before the final exam to check your preparedness and tune your revision strategy (hyperparameter tuning & model selection).
- **Test Data = The Final Board Exam:** A completely unseen, held-out dataset evaluated strictly once. If a professor reveals exam questions in homework assignments, scoring 100% does not prove mastery — it is memorization (overfitting/cheating). Similarly, a model must NEVER see test data during training!

[KEY CONCEPT] Production Drift: Data Drift vs. Concept Drift

Deploying an ML model is not the end of the journey. In production, real-world data distributions evolve over time:

- **Data Drift (Covariate Shift):** The input feature distribution $P(X)$ changes, while $P(Y|X)$ remains constant. (Example: In housing, users begin listing much older rural houses that were never present in urban training data).
- **Concept Drift:** The relationship between features and labels $P(Y|X)$ changes. (Example: Consumer purchasing behavior shifted drastically during COVID-19 lockdowns; what people bought at a given income level changed completely). When drift occurs, models must be retrained on fresh data.

7. LINEAR ALGEBRA PREREQUISITES FOR MLF

A student specifically asked what Linear Algebra topics must be revised for MLF. The instructor outlined the prerequisite toolkit that will be used starting Week 2:

Area	Core Concepts Required in MLF	Relevance to Machine Learning
Vector Spaces	Subspaces, Linear Combinations, Span, Linear Independence, Basis, Dimension.	Feature spaces, degrees of freedom, identifying redundant (collinear) features.
Inner Products & Orthogonality	Dot product, Euclidean norm, Cauchy-Schwarz inequality, Orthogonal vectors, Orthonormal bases.	Measuring cosine similarity, orthogonal projections (crucial for Least Squares Linear Regression in Week 4).
Fundamental Subspaces	Column Space $C(A)$, Null Space $N(A)$, Row Space, Matrix Rank, Invertibility.	Determining whether linear systems $X\mathbf{w} = \mathbf{y}$ have exact, infinite, or approximate solutions.
Eigenvalues & Spectral Theory	Eigenvalues, Eigenvectors, Symmetric matrices, Orthogonal matrices, Positive semi-definite matrices.	Principal Component Analysis (PCA), Singular Value Decomposition (SVD), Covariance matrix analysis.

8. RECOMMENDED TEXTBOOKS & STUDY STRATEGY FOR MLF

8.1 Authoritative Textbooks Recommended by Karthik Sir

- **Introduction to Linear Algebra** — *Gilbert Strang* (MIT). The primary textbook followed by the professor in the course video lectures.
- **Linear Algebra and Its Applications** — *Gilbert Strang*. Excellent companion focusing on geometric applications and matrix factorizations.
- **Linear Algebra with Applications** — *Johnson, Riess, & Arnold*. Classic, rigorous, and highly recommended by Karthik Sir for matrix algebra problem-solving.
- **Pattern Recognition and Machine Learning (PRML)** — *Christopher M. Bishop*. Freely available online; standard graduate-level reference for probabilistic and algebraic ML.

[EXAM & STRATEGY TIP] Karthik Sir's Actionable Strategy to Pass & Excel in MLF

A student repeating MLF asked: "What is the best way to pass and score well in this subject? It requires tremendous effort."

Karthik Sir gave three specific, non-negotiable guidelines:

- 1. Solve Every Single Assignment Problem Yourself:** Do not rely on leaked solutions or peer answers. The algebraic mechanics must flow through your own pen.
- 2. Master ALL Previous Year Questions (PYQs):** Exam questions frequently test standard archetypes and problem structures. By mastering PYQs from all prior terms, you ensure that even when an exam problem introduces a novel twist, you already recognize the standard template.
- 3. Maintain Geometric Intuition:** Never treat matrix equations as dry arithmetic. When computing projections, null spaces, or eigenvalues, visualize the geometric subspace.

9. INTERACTIVE Q&A; HIGHLIGHTS & EDGE CASES FROM SESSION

- Q1: Why do we use squared error instead of absolute error in regression?**
Answer: Absolute error ($|e|$) has a non-differentiable sharp kink at zero, which impedes standard gradient-based optimization algorithms. Squared error (e^2) is smooth, everywhere differentiable, and forms a convex parabolic surface whose gradient ∇L can be computed cleanly via multivariate calculus.
- Q2: Is the zero-one loss applicable to multi-class classification?**
Answer: Yes, conceptually 0-1 loss applies to any discrete classification task: $L = 1$ if the predicted class does not equal the true class, and 0 if correct. However, for optimization, we use **Categorical Cross-Entropy** via the Softmax function rather than binary logistic loss.
- Q3: Does the learning algorithm discover features, or do humans?**
Answer: In classical ML (MLF/MLT), feature engineering is primarily conducted by human data scientists who extract domain-specific attributes. In Deep Learning, feature extraction is automated via hierarchical representation learning.
- Q4: Can we predict confidence intervals in regression?**
Answer: While standard deterministic linear regression outputs point predictions $y_{\text{pred}} \in \mathbf{R}$, one can adopt a probabilistic perspective (Gaussian noise assumption: $y \sim N(\mathbf{w}^T \mathbf{x}, \sigma^2)$), enabling prediction intervals.

10. MASTER MATHEMATICAL FORMULA & CONCEPT CHEATSHEET

Concept	Mathematical Formula / Definition	Key Properties & Usage
Design Matrix	$X \in \mathbf{R}^{n \times d}, \mathbf{x}_i \in \mathbf{R}^d$	n instances (rows), d features (columns). The fundamental dataset representation.
Linear Model	$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$	Parametric linear hypothesis with weight vector $\mathbf{w}$ and bias/intercept b .
Mean Squared Error	$\text{MSE} = (1/n) \sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2$	Standard regression loss. Smooth, everywhere differentiable, convex quadratic surface.
Zero-One Loss	$L_{0-1}(y, y_{\text{pred}}) = I(y \neq y_{\text{pred}})$	Exact misclassification metric. Non-convex, gradient = 0 almost everywhere.
Hinge Loss	$\max(0, 1 - y \cdot g(\mathbf{x}))$	Convex surrogate used in Support Vector Machines (SVM). Enforces margin ≥ 1 .
Logistic Loss	$\log(1 + \exp(-y \cdot g(\mathbf{x})))$	Convex surrogate used in Logistic Regression. Smooth, probabilistic basis.
Categorical Cross-Entropy	$-\sum_{k=1}^K y_k \log(p_k)$	Multi-class classification loss function paired with Softmax activation.
Train / Val / Test	$D_{\text{train}} \cap D_{\text{val}} \cap D_{\text{test}} = \emptyset$	Train (fit parameters), Val (tune hyperparams), Test (held-out final evaluation).

These master lecture notes synthesize the live session of Week 1, Session 1 of MLF taught by Karthik Thiagarajan at IIT Madras. All mathematical derivations, examples, pedagogical analogies, and student discussions have been preserved and expanded for academic rigor and exam preparation.